

Aim: Naïve Bayes.**1.Creating a random dataset manually.****CODE:**

```
import pandas as pd

data = {
    'Age': ['Youth', 'Youth', 'Middle_Aged', 'Senior','Senior', 'Senior', 'Middle_Aged', 'Youth', 'Youth','Senior', 'Youth', 'Middle_Aged', 'Middle_Aged',
    'Senior'],
    'Income': ['High', 'High', 'High', 'Medium', 'Low','Low', 'Low', 'Medium', 'Low', 'Medium', 'Medium',
    'Medium', 'High', 'Medium'],
    'Student': ['No', 'No', 'No', 'No', 'Yes', 'Yes','Yes', 'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes', 'No'],
    'Credit_Rating': ['Fair', 'Excellent', 'Fair','Fair', 'Fair', 'Excellent', 'Excellent', 'Fair','Fair', 'Fair', 'Excellent', 'Excellent', 'Fair',
    'Excellent'],
    'Buys_Computer': ['No', 'No', 'Yes', 'Yes', 'Yes','No', 'Yes', 'No', 'Yes', 'Yes', 'Yes', 'Yes', 'Yes','No']
}
df = pd.DataFrame(data)
df
```

OUTPUT:

	Age	Income	Student	Credit_Rating	Buys_Computer
0	Youth	High	No	Fair	No
1	Youth	High	No	Excellent	No
2	Middle_Aged	High	No	Fair	Yes
3	Senior	Medium	No	Fair	Yes
4	Senior	Low	Yes	Fair	Yes
5	Senior	Low	Yes	Excellent	No
6	Middle_Aged	Low	Yes	Excellent	Yes
7	Youth	Medium	No	Fair	No
8	Youth	Low	Yes	Fair	Yes
9	Senior	Medium	Yes	Fair	Yes
10	Youth	Medium	Yes	Excellent	Yes
11	Middle_Aged	Medium	No	Excellent	Yes
12	Middle_Aged	High	Yes	Fair	Yes
13	Senior	Medium	No	Excellent	No

2.Converting labels to numeric.**CODE:**

```
from sklearn.preprocessing import LabelEncoder
df_encoded = df.copy()
le = LabelEncoder()
for column in df_encoded.columns:
    df_encoded[column] = le.fit_transform(df_encoded[column])
df_encoded
```

OUTPUT:

	Age	Income	Student	Credit_Rating	Buys_Computer
0	2	0	0	1	0
1	2	0	0	0	0
2	0	0	0	1	1
3	1	2	0	1	1
4	1	1	1	1	1
5	1	1	1	0	0
6	0	1	1	0	1
7	2	2	0	1	0
8	2	1	1	1	1
9	1	2	1	1	1
10	2	2	1	0	1
11	0	2	0	0	1
12	0	0	1	1	1
13	1	2	0	0	0

3.Initializing the model, training and testing the datasets.

CODE:

```
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report

X = df_encoded.drop('Buys_Computer', axis =1)
y = df_encoded['Buys_Computer']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.3, random_state = 42)
model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
y_pred
```

OUTPUT:

```
array([1, 0, 0, 1, 1])
```

4.Evaluating the model.

CODE:

```
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

OUTPUT:

```
Accuracy: 0.6
```

```
Classification Report:
```

	precision	recall	f1-score	support
0	0.50	0.50	0.50	2
1	0.67	0.67	0.67	3
accuracy			0.60	5
macro avg	0.58	0.58	0.58	5
weighted avg	0.60	0.60	0.60	5

Bonus: Predict if a Middle_Aged, Medium_Income, Student, Fair credit_rating person buy a computer.

CODE:

```
sample = pd.DataFrame({ 'Age' : [le.fit(df['Age']).transform(['Middle_Aged'])[0]],
                        'Income' : [le.fit(df['Income']).transform(['Medium'])[0]],
                        'Student' : [le.fit(df['Student']).transform(['Yes'])[0]],
                        'Credit_Rating' : [le.fit(df['Credit_Rating']).transform(['Fair'])[0]]
                      })

model.predict(sample)
```

OUTPUT:

```
array([1])
```

CODE:

```

memes = {
    'Caption': [
        'Me: I'll sleep early. Also me at 3AM: Let's learn quantum physics',
        'Sunsets are proof that there's beauty in endings',
        '404: Motivation not found',
        'I love deadlines. I like the whooshing sound they make as they fly by.',
        'Peace, love, and gratitude',
        'When the code finally works and you don't know why',
        'A smile is the best makeup',
        'Git commit -m "final final version for real this time"',
        'Nature is the art of God',
        'Me trying to fix a bug by randomly typing things'],
    'Funny': [1, 0, 1, 1, 0, 1, 0, 1, 0, 1]}

df = pd.DataFrame(memes)

from wordcloud import WordCloud
import matplotlib.pyplot as plt

funny_text = ' '.join(df[df['Funny'] == 1]['Caption'])
not_funny_text = ' '.join(df[df['Funny'] == 0]['Caption'])

fig, axs = plt.subplots(1, 2, figsize=(14, 6))
axs[0].imshow(WordCloud(background_color='white').generate(funny_text))
axs[0].set_title("😄 Funny Meme Words", fontsize=16)
axs[0].axis('off')
axs[1].imshow(WordCloud(background_color='white').generate(not_funny_text))
axs[1].set_title("😞 Not Funny Meme Words", fontsize=16)
axs[1].axis('off')
plt.tight_layout()
plt.show()

```

OUTPUT:



Aim: Using Naïve Bayes for weather prediction.

CODE:

```
import pandas as pd
data = {
    'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain', 'Rain',
               'Overcast', 'Sunny', 'Sunny', 'Rain', 'Sunny', 'Overcast',
               'Overcast', 'Rain'],
    'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool', 'Cool',
                   'Cool', 'Mild', 'Cool', 'Mild', 'Mild', 'Mild',
                   'Hot', 'Mild'],
    'Humidity': ['High', 'High', 'High', 'High', 'Normal', 'Normal',
                'Normal', 'High', 'Normal', 'Normal', 'Normal', 'High',
                'Normal', 'High'],
    'Wind': ['Weak', 'Strong', 'Weak', 'Weak', 'Weak', 'Strong',
            'Strong', 'Weak', 'Weak', 'Weak', 'Strong', 'Strong',
            'Weak', 'Strong'],
    'PlayTennis': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No',
                  'Yes', 'No', 'Yes', 'Yes', 'Yes', 'Yes',
                  'Yes', 'No']
}

df = pd.DataFrame(data)
print("Original Dataset:\n", df)

from sklearn.preprocessing import LabelEncoder
df_encoded = df.copy()
le = LabelEncoder()
for column in df_encoded.columns:
    df_encoded[column] = le.fit_transform(df_encoded[column])

print("\nEncoded Dataset:\n", df_encoded)

from sklearn.model_selection import train_test_split
X = df_encoded.drop('PlayTennis', axis=1)
y = df_encoded['PlayTennis']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report

model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("\nPredictions:", y_pred)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

OUTPUT:

Original Dataset:

	Outlook	Temperature	Humidity	Wind	PlayTennis
0	Sunny	Hot	High	Weak	No
1	Sunny	Hot	High	Strong	No
2	Overcast	Hot	High	Weak	Yes
3	Rain	Mild	High	Weak	Yes
4	Rain	Cool	Normal	Weak	Yes
5	Rain	Cool	Normal	Strong	No
6	Overcast	Cool	Normal	Strong	Yes
7	Sunny	Mild	High	Weak	No
8	Sunny	Cool	Normal	Weak	Yes
9	Rain	Mild	Normal	Weak	Yes
10	Sunny	Mild	Normal	Strong	Yes
11	Overcast	Mild	High	Strong	Yes
12	Overcast	Hot	Normal	Weak	Yes
13	Rain	Mild	High	Strong	No

Encoded Dataset:

	Outlook	Temperature	Humidity	Wind	PlayTennis
0	2	1	0	1	0
1	2	1	0	0	0
2	0	1	0	1	1
3	1	2	0	1	1
4	1	0	1	1	1
5	1	0	1	0	0
6	0	0	1	0	1
7	2	2	0	1	0
8	2	0	1	1	1
9	1	2	1	1	1
10	2	2	1	0	1
11	0	2	0	0	1
12	0	1	1	1	1
13	1	2	0	0	0

Predictions: [1 0 0 1 1]

Accuracy: 0.6

Classification Report:

	precision	recall	f1-score	support
0	0.50	0.50	0.50	2
1	0.67	0.67	0.67	3
accuracy			0.60	5
macro avg	0.58	0.58	0.58	5
weighted avg	0.60	0.60	0.60	5